

A type system framework for safe (a)synchronous programming

João Mota - jd.mota@campus.fct.unl.pt

Advisers: António Ravara and Marco Giunti

NOVA LINCS and NOVA School of Science & Technology

Producer + Consumer in JS

```
const queue = new Queue();
```

```
async function producer(amount) {  
  while (amount-- > 0) {  
    const data = await otherTask1();  
    queue.add(data);  
  }  
}
```

```
async function consumer(amount) {  
  while (amount-- > 0) {  
    while (queue.isEmpty()) await null;  
  
    const data = queue.take();  
    await otherTask2(data);  
  }  
}
```

```
await Promise.all([ producer(10), consumer(10) ]);
```

Producer + Consumer in Java

```
MyQueue queue = new MyQueue();
```

```
Thread producer = new Thread(() -> {  
    int amount = 10;  
    while (amount-- > 0) {  
        Data data = otherTask1();  
        queue.add(data);  
    }  
});
```

```
Thread consumer = new Thread(() -> {  
    int amount = 10;  
    while (amount-- > 0) {  
        while (queue.isEmpty()) queue.await();  
  
        Data data = queue.take();  
        otherTask2(data);  
    }  
});
```

```
producer.start(); consumer.start();
```

Problem

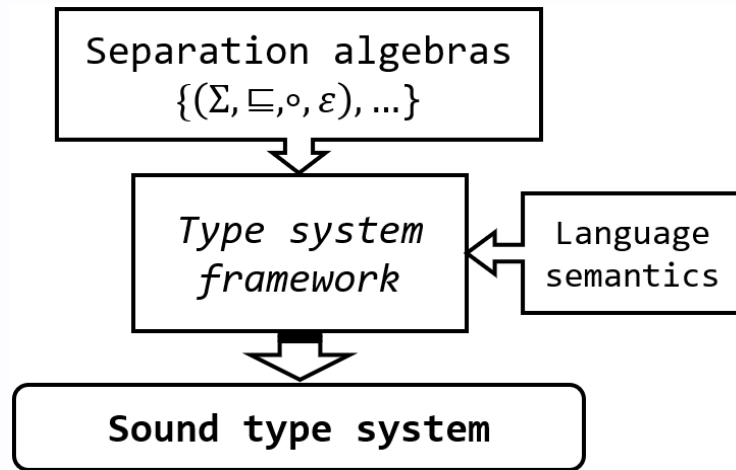
- Languages have *different concurrency models*
 - Single-threaded asynchrony (e.g., in JS)
 - Multi-threading (e.g., in Java)
- How to check **memory safety**, **thread-safety**, and **avoid memory leaks**?
 - No distinction between *thread-local* and *thread-shared* data forces access to shared data to be *synchronized*
 - Fully *affine* logics *leak resources*
 - Fully *linear* logics are too restrictive

Avoid "the next 700 type systems"

Create a **type system framework**
parametric over *separation algebras*.

Taking inspiration from Iris^[1]

- easing the creation of new type systems
- guarantee memory safety, thread-safety, and avoid memory leaks, in asynchronous languages



1. <https://iris-project.org/> ↩

Affine and linear resources

- **affine** resources can be "dropped" (and be garbage collected)
- **linear** resources need to be explicitly disposed
even without `free()`, these are useful to avoid memory leaks

E.g., queue as a linear resource until it is empty

E.g., a socket as a linear resource until calling `close()`

Thread-local and thread-shared resources

- **thread-local** resources may be accessed without synchronization
- **thread-shared** resources need synchronization

E.g., queue as a thread-local resource in JS

E.g., queue as a thread-shared resource in Java

Non-frame-preserving updates

- **Non-frame-preserving updates:** allow to *temporarily break assumptions* made by thread-local resources and permit one to restore consistency before changes become visible to others
E.g., between `await` keywords in JS
- Allow reasoning on **separation degrees** (e.g., strong vs weak)

E.g., allow the producer and consumer in single-threaded asynchronous code to access the queue in turns (without locks)

Assumptions

A linear separation logic with fractional permissions and two notions of separation [\[1\]](#)

Weak (or fictional) separation: $x \mapsto^{\frac{1}{2}} n \circledast y \mapsto^{\frac{1}{2}} m$

Strong separation: $x \mapsto^{\frac{1}{2}} n * y \mapsto^{\frac{1}{2}} m$ (implies that $x \neq y$)

Weakening: $P * Q \models P \circledast Q$

-
1. Brotherston et al. Reasoning over permissions regions in concurrent separation logic. CAV 2020 [↩](#)

A shared queue

$\text{Queue}(q)$ asserts ownership of a queue referenced by variable q

A producer owns $\text{Queue}(q)^{\frac{1}{2}}$, and a consumer owns $\text{Queue}(q)^{\frac{1}{2}}$

Modularity: prove each party correct, and then compose both proofs.

Normally, $\frac{1}{2}$ would only allow for read access...

but if we support **non-frame-preserving updates**, we can do writes.

A non-frame-preserving update

$$\begin{aligned} \text{Queue}(q)^{\frac{1}{2}} &\xRightarrow{\text{focus}} \{\mathbf{Focused}\}_i \\ &\quad [\text{Guarantee}]_i \\ \{\mathbf{Queue}(q)^{\frac{1}{2}}\}_i \otimes [\text{Queue}(q)^{\frac{1}{2}}]_i &\xRightarrow{\text{unfold} + \text{mutate}} \\ \{\dots\}_i \otimes [\text{Queue}(q)^{\frac{1}{2}}]_i &\xRightarrow{\text{restore} + \text{fold}} \\ \{\mathbf{Queue}(q)^{\frac{1}{2}}\}_i \otimes [\text{Queue}(q)^{\frac{1}{2}}]_i &\xRightarrow{\text{defocus}} \\ \text{Queue}(q)^{\frac{1}{2}} \end{aligned}$$

Inspired in "Rely-Guarantee Protocols" by Militão et al. (ECOOP 2014 & Ph.D. Thesis)

Framing with *

$$\text{Queue}(q)^{\frac{1}{2}} * F \xRightarrow{\text{focus}}$$

{Focused}_i

[Guarantee]_i

$$(\{\mathbf{Queue}(\mathbf{q})^{\frac{1}{2}}\}_i \otimes [\text{Queue}(q)^{\frac{1}{2}}]_i) * F \xRightarrow{\text{unfold} + \text{mutate}}$$

$$(\{\dots\}_i \otimes [\text{Queue}(q)^{\frac{1}{2}}]_i) * F \xRightarrow{\text{restore} + \text{fold}}$$

$$(\{\mathbf{Queue}(\mathbf{q})^{\frac{1}{2}}\}_i \otimes [\text{Queue}(q)^{\frac{1}{2}}]_i) * F \xRightarrow{\text{defocus}}$$

$$\text{Queue}(q)^{\frac{1}{2}} * F$$

Inspired in "Rely-Guarantee Protocols" by Militão et al. (ECOOP 2014 & Ph.D. Thesis)

Framing with $\circledast$

$\{\mathbf{Focused}\}_i$

$[\mathbf{Guarantee}]_i$

$[\mathbf{Disabled}]_{\{i\}}$

$$\text{Queue}(q)^{\frac{1}{2}} \circledast \text{Queue}(q)^{\frac{1}{2}} \xRightarrow{\text{focus}}$$

$$\{\mathbf{Queue}(\mathbf{q})^{\frac{1}{2}}\}_i \circledast [\mathbf{Queue}(q)^{\frac{1}{2}}]_i \circledast [\text{Queue}(q)^{\frac{1}{2}}]_{\{i\}} \xRightarrow{\text{unfold}+\text{mutate}}$$

$$\{\dots\}_i \circledast [\mathbf{Queue}(q)^{\frac{1}{2}}]_i \circledast [\text{Queue}(q)^{\frac{1}{2}}]_{\{i\}} \xRightarrow{\text{restore}+\text{fold}}$$

$$\{\mathbf{Queue}(\mathbf{q})^{\frac{1}{2}}\}_i \circledast [\mathbf{Queue}(q)^{\frac{1}{2}}]_i \circledast [\text{Queue}(q)^{\frac{1}{2}}]_{\{i\}} \xRightarrow{\text{defocus}}$$

$$\text{Queue}(q)^{\frac{1}{2}} \circledast \text{Queue}(q)^{\frac{1}{2}}$$

Inspired in "Rely-Guarantee Protocols" by Militão et al. (ECOOP 2014 & Ph.D. Thesis)

(Tentative) Rules

$$P \vdash \hookrightarrow (\{\mathbf{P}\}_i \otimes [P]_i) \quad \text{Focus}$$

$$\{\mathbf{P}\}_i \otimes [P]_i \vdash \hookrightarrow P \quad \text{Defocus}$$

$\hookrightarrow$ is a novel modality to support our *non-frame-preserving updates*

(Tentative) Rules

$$\frac{P \vdash \Rightarrow_* Q}{\{\mathbf{P}\}_i \circledast [\mathbf{G}]_i \vdash \hookrightarrow (\{\mathbf{Q}\}_i \circledast [\mathbf{G}]_i)} \quad \text{Update Focused}$$

$\Rightarrow_*$ is an update modality (like the Iris' one) but only frame-preserving for *strongly separated resources* (i.e., with $*$)

$$[[\Rightarrow_* P]] \triangleq \lambda a. \forall f. \checkmark (a \cdot f) \wedge \mathbf{a} * \mathbf{f} \implies \exists b. \checkmark (b \cdot f) \wedge \mathbf{b} * \mathbf{f} \wedge [[P]](b)$$

(Tentative) Rules

$$(\hookrightarrow P) * F \vdash \hookrightarrow (P * F)$$

$$\frac{\{P\} e \{Q\}}{\{P * F\} e \{Q * F\}} \quad \text{Frame } *$$

$$(\hookrightarrow P) \circledast\!-\! F \vdash \hookrightarrow (P \circledast\!-\! F)$$

$$\frac{\{P\} e \{Q\}}{\{P \circledast\!-\! F\} e \{Q \circledast\!-\! F\}} \quad \text{Frame } \circledast\!-\!$$

$P \circledast\!-\! F$ extends the footprint P with a frame F , automatically "disabling" resources in F that may interfere [\[1\]](#)

1. Borrowed from "Rely-Guarantee Protocols" by Militão et al. (ECOOP 2014) [↩](#)

(Tentative) Derivation example

$$\{Q(q)^{\frac{1}{2}}\} \text{ e } \{\{\dots\}_i \otimes [Q(q)^{\frac{1}{2}}]_i\}$$

$$\{Q(q)^{\frac{1}{2}} \otimes - Q(q)^{\frac{1}{2}}\} \text{ e } \{(\{\dots\}_i \otimes [Q(q)^{\frac{1}{2}}]_i) \otimes - Q(q)^{\frac{1}{2}}\} \text{ (Frame } \otimes -)$$

$$\{Q(q)^{\frac{1}{2}} \otimes Q(q)^{\frac{1}{2}}\} \text{ e } \{\{\dots\}_i \otimes [Q(q)^{\frac{1}{2}}]_i \otimes [Q(q)^{\frac{1}{2}}]_{\{i\}}\}$$

Note: if we had (interfering) active guarantees on the right of $\otimes -$, these would also get disabled, allowing for nesting.

Inspired in "Rely-Guarantee Protocols" by Militão et al. (ECOOP 2014 & Ph.D. Thesis)

Why would this be sound?

Non-frame-preserving updates should only...

- ...apply to thread-local resources
- ...occur, e.g., between `await` keywords
 - preventing the existence of opened guarantees before there is a context switch
 - allowing any asynchronous task to trust the resources it holds when it recovers control
 - permitting modular and independent proofs

Preliminary (mechanized) results

A core sep. logic framework parametric over a separation algebra

- **Simplified setting:** first-order logic and no step-indexing
- Separation algebras equipped with a **preorder** [\[1\]](#) [\[2\]](#)
 - If $a \sqsubseteq b$, then if P holds for a , it also holds for b
 - If $\varepsilon \sqsubseteq a$, then a is completely affine

-
1. Cao et al. Bringing Order to the Separation Logic Jungle. APLAS 2017 [↩](#)
 2. Krebbers et al. MoSeL: a general, extensible modal framework for interactive proofs in separation logic. ICFP 2018 [↩](#)

Preliminary (mechanized) results

A core sep. logic framework parametric over a separation algebra

- Strong and weak separation ^[1] generalized to...
 $\otimes/*$ distribution: $(A \otimes B) * (C \otimes D) \models (A * C) \otimes (B * D)$
- ...**separation degrees**, defined as relations over resources
 - degrees distribution:
If $d \subseteq d'$ then $(A *_{d'} B) *_d (C *_{d'} D) \models (A *_d C) *_{d'} (B *_d D)$
 - d -update modality: $\models_d$
Frame rule: $(\models_d A) *_d F \models \models_d (A *_d F)$

1. Brotherston et al. Reasoning over permissions regions in concurrent separation logic.
CAV 2020 [↩](#)

Work in progress...

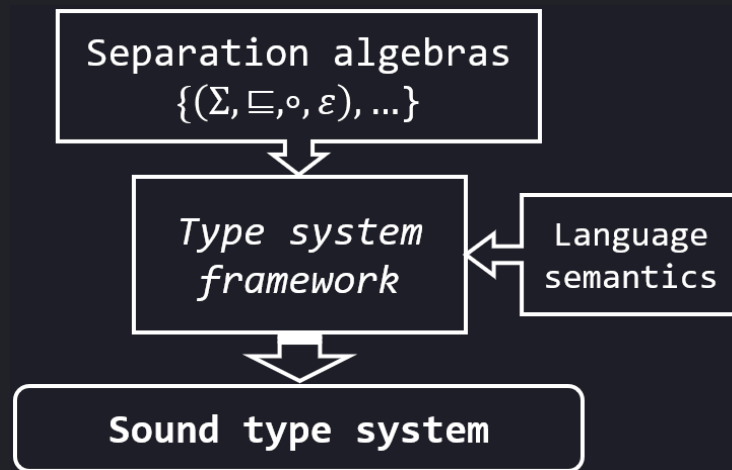
- Devise the semantic model for the **update modality** $\hookrightarrow$ supporting **non-frame-preserving updates**
- Define an adequate **weakest-precondition** for the verification of a core language with asynchronous tasks (similar to JS)
- Derive some Hoare-triple style rules to show the usefulness of the approach

Long term plan: extract a decidable fragment to produce a type system framework, all mechanized in Rocq

Thank you!

jd.mota@campus.fct.unl.pt

jd.mota.github.io



Acknowledgements: UID/04516/NOVA Laboratory for Computer Science and Informatics (NOVA LINCS) with the financial support of FCT.IP (2021.05297.BD).